

TP 3 : Déploiement sur AWS

Infrastructure as Code avec Terraform - Partie 1

Travail Pratique N°3



Saad KORCHI

Étudiant en Cybersécurité

Encadré par :

Pr. AMAMOU Ahmed

EIDIA Cybersécurité

SOMMAIRE

1	Introduction et Contexte du Projet	3
1.1	Objectif du Projet	3
1.2	Architecture Cible	3
1.3	Choix Technologiques	3
2	Mise en Place de l'Environnement de Travail	3
2.1	Installation des Outils	3
2.2	Configuration des Identifiants AWS	4
3	Conception de l'Infrastructure avec Terraform	4
3.1	Structure du Code Terraform	4
3.2	Ressources AWS Provisionnées	4
4	Gestion de l'État Terraform	4
4.1	Problématique	4
4.2	Solutions Envisagées	4
4.3	Implémentation Retenue	5
	Captures d'écran	6

1 Introduction et Contexte du Projet

1.1 Objectif du Projet

Objectif principal

L'objectif principal de ce projet est de déployer une application web d'upload de fichiers sur l'infrastructure cloud d'**Amazon Web Services (AWS)**. L'application, développée en Node.js, permet aux utilisateurs de téléverser des fichiers qui sont ensuite stockés de manière sécurisée.

1.2 Architecture Cible

L'architecture vise à être scalable, hautement disponible et entièrement automatisée. Elle repose sur les services AWS suivants :

- **Amazon ECS Fargate** : Pour exécuter l'application conteneurisée sans avoir à gérer les serveurs.
- **Application Load Balancer (ALB)** : Pour distribuer le trafic entrant entre les différentes instances de l'application.
- **Amazon ECR** : Pour stocker les images Docker de l'application.
- **Amazon VPC** : Pour isoler le réseau et garantir la sécurité.
- **AWS IAM** : Pour gérer les permissions et l'authentification.

1.3 Choix Technologiques

- **Terraform** : Pour l'Infrastructure as Code (IaC). Ce choix a été motivé par sa capacité à gérer l'état de l'infrastructure, sa syntaxe déclarative et sa compatibilité multi-cloud.
- **GitLab CI/CD** : Pour l'intégration et le déploiement continus. GitLab offre une intégration native avec son registre de conteneurs.
- **AWS** : Comme fournisseur cloud, offrant une large gamme de services managés.

2 Mise en Place de l'Environnement de Travail

2.1 Installation des Outils

La première étape a consisté à préparer l'environnement de développement local sur Windows :

- **Terraform** : Installation via le gestionnaire de paquets winget
- **AWS CLI** : Installation pour interagir avec les services AWS
- **Docker Desktop** : Installation et activation du support WSL 2

2.2 Configuration des Identifiants AWS

Pour permettre aux outils de communiquer avec AWS, un utilisateur IAM nommé `adam` a été créé :

- Des clés d'accès (Access Key ID et Secret Access Key) ont été générées
- La commande `aws configure` a été utilisée pour enregistrer ces identifiants

3 Conception de l'Infrastructure avec Terraform

3.1 Structure du Code Terraform

Le code Terraform a été organisé de manière modulaire :

```
1 terraform/  
2     main.tf           # Ressources principales  
3     variables.tf      # Declaration des variables  
4     outputs.tf        # Definition des sorties (URL, etc.)
```

3.2 Ressources AWS Provisionnées

Le fichier `main.tf` décrit l'ensemble des ressources nécessaires :

- **VPC et Réseau** : Création d'un VPC avec deux sous-réseaux publics, d'une passerelle Internet et d'une table de routage
- **Sécurité** : Définition de groupes de sécurité pour l'ALB et les tâches ECS
- **Registre d'Images** : Création d'un dépôt ECR pour stocker les images Docker
- **Équilibreur de Charge** : Mise en place d'un ALB avec target group et listener
- **Orchestration** : Configuration d'un cluster ECS, d'une définition de tâche et d'un service ECS
- **Logs** : Création d'un groupe de logs CloudWatch
- **Permissions** : Création d'un rôle IAM pour les tâches ECS

4 Gestion de l'État Terraform

4.1 Problématique

La gestion de l'état (`terraform.tfstate`) est cruciale en équipe. Cet état doit être partagé et verrouillé pour éviter les conflits.

4.2 Solutions Envisagées

- **Backend local** : Inadapté pour le travail collaboratif
- **Backend S3** : Robuste mais nécessite bucket S3 et table DynamoDB
- **Backend HTTP GitLab** : Solution native et intégrée

4.3 Implémentation Retenue

La solution du backend HTTP GitLab a été choisie :

```
1 terraform {  
2   backend "http" {  
3   }  
4 }
```

Fin du rapport TP3 - La suite sera traitée dans le TP4

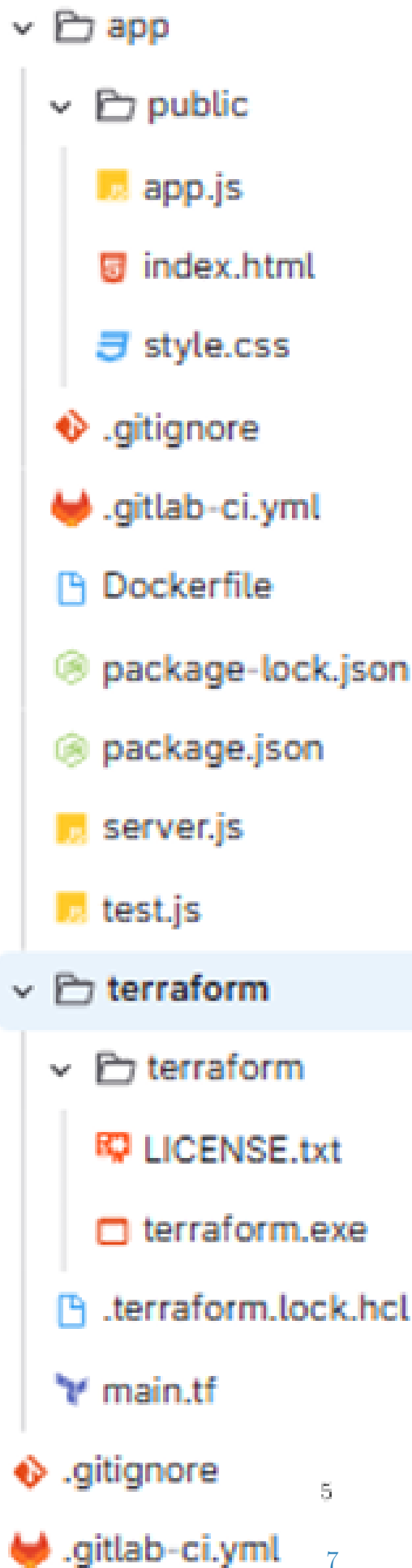
Captures d'écran

```
PS C:\Users\pc> terraform --version
Terraform v1.14.5
on windows_amd64
PS C:\Users\pc> aws --version
aws-cli/2.33.22 Python/3.11.11 windows/10 exe/AMD64
PS C:\Users\pc> docker --version
Docker version 29.2.1, build a3c7197
PS C:\Users\pc>
```

FIGURE 1 – Figure 1

Récapitulatif		
ARN  arn:aws:iam::831492176060:user/adam	Accès par console  Activé sans l'authentification MFA	Clé d'accès 1 AKIA4DGGGLXC6FOTWLU6P - Active  Utilisé aujourd'hui. 4 jours ancien.
Création February 16, 2026, 17:40 (UTC)	Dernière connexion à la console  Aujourd'hui	Clé d'accès 2 Créer une clé d'accès

FIGURE 2 – Figure 2



5

7

FIGURE 3 – Figure 3

```

#####
# Networking (single public vpc)
#####
resource "aws_vpc" "main" {
  cidr_block      = "10.0.0.0/16"
  enable_dns_hostnames = true
  enable_dns_support = true

  tags = {
    Name = "${var.app_name}-vpc"
  }
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "${var.app_name}-igw"
  }
}

resource "aws_subnet" "public_a" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.0.0/24"
  availability_zone = data.aws_availability_zones.available.names[0]
  map_public_ip_on_launch = true

  tags = {
    Name = "${var.app_name}-public-a"
  }
}

resource "aws_subnet" "public_b" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.0.0/24"
  availability_zone = data.aws_availability_zones.available.names[1]
  map_public_ip_on_launch = true

  tags = {
    Name = "${var.app_name}-public-b"
  }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }

  tags = {
    Name = "${var.app_name}-public-rt"
  }
}

resource "aws_route_table_association" "public_a" {
  subnet_id = aws_subnet.public_a.id
  route_table_id = aws_route_table.public.id
}

resource "aws_route_table_association" "public_b" {
  subnet_id = aws_subnet.public_b.id
  route_table_id = aws_route_table.public.id
}

```

FIGURE 4 – Figure 4